BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO ASSIGNMENT 03

PHÁT TRIỂN HỆ THỐNG THÔNG MINH

Giảng viên: **Trần Đình Quế**

Sinh viên: **Phạm Hồng Sơn**

Mã sinh viên: **B23DCCN723**

Nhóm lớp: **06**

Hà Nội – 2026

MỤC LỤC

PHỤ LỤC - TÀI NGUYÊN VÀ MÃ NGUỒN DỰ ÁN **3**
1. Mã nguồn hệ thống (GitHub Repository) 3
2. Kết quả thực nghiệm và Tài liệu đính kèm (Google Drive) 3
PHẦN I - GIỚI THIỆU CHUNG **5**
1.1. Mục tiêu 5
1.2. Tổng quan về bài toán và tập dữ liệu 5
1.2.1. Bài toán đặt ra 5
1.2.2. Tập dữ liệu 6
PHẦN II - CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH KỸ THUẬT **6**
2.1. Biến đổi kích thước Tensor quan các lớp 6
2.2. Cơ chế tối ưu hóa và hàm mất mát 8
2.2.1. Hàm mất mắt Binary Cross-Entropy 8
2.2.2 Cơ chế tối ưu hóa bằng Gradient Descent 8
2.3. Giải thích mã nguồn và vòng lặp huấn luyện 9
2.4. Phân biệt chế độ huấn luyện và đánh giá 9
2.5. Thuật toán lan truyền ngược 10
PHẦN III: THIẾT KẾ THỰC NGHIỆM VÀ KẾT QUẢ **12**
3.1. Thí nghiệm 1: Baseline MLP 12
3.1.1. Cấu trúc mạng: Sơ đồ kiến trúc và tổng số lượng tham số huấn luyện (parameters). 12
3.1.2. Cấu hình thực nghiệm 13
3.1.3. Quá trình hội tụ: Đồ thị đường cong Loss và Accuracy theo epoch. 13
3.1.4. Đánh giá hiệu năng: Các chỉ số trên tập kiểm tra và Ma trận nhầm lẫn. 13
3.2. Thí nghiệm 2: Deeper MLP 13
3.2.1. Cấu trúc mạng: Sơ đồ kiến trúc và số lượng tham số. 13
3.2.2. Cấu hình thực nghiệm. 14
3.2.3. Đồ thị đường cong hội tụ (Train vs. Validation). 14
3.3. Thí nghiệm 3: Khảo sát quá trình Tối ưu hóa 14
3.3.1. Thiết lập đối chứng: So sánh giữa 2 tốc độ học khác nhau (hoặc 2 bộ tối ưu hóa khác nhau, ví dụ: Adam vs. SGD). 14
3.3.2. Đồ thị so sánh: Tốc độ hội tụ của hàm loss và độ chính xác giữa hai cấu hình. 15
3.3.3. Phân tích kết quả: Ảnh hưởng của tham số tối ưu hóa đến hiệu năng cuối cùng. 15
3.3.3.1. Về Tốc độ hội tụ (Convergence Speed): 15
3.3.3.2. Về tính Ổn định và Hiệu năng cuối cùng (Final Performance): 15
PHẦN IV: PHÂN TÍCH CHUYÊN SÂU VÀ THẢO LUẬN KHOA HỌC **16**
4.1. Phân tích hiện tượng Overfitting và Underfitting: 16
4.1.1. Nhận diện hiện tượng dựa trên sự phân kỳ của các đường cong: 16

4.1.2. Đề xuất các giải pháp khắc phục tiềm năng: 16
4.2. Khảo sát Không không gian Đặc trưng học được (Inspecting Learned Representation): 17
4.2.1. Cách thức trích xuất đầu ra của tầng ẩn: 17
4.2.2. Phân tích sự thay đổi số chiều dữ liệu (Dimensionality Change): 17
4.2.3. Đánh giá khả năng phân tách trong không gian mới: 18
4.3. So sánh và kết luận thực nghiệm tổng thể: 18
4.3.1. Bảng tổng hợp kết quả của 3 thí nghiệm: 18
4.3.2. Đánh giá mối quan hệ khoa học: 19
PHẦN V: TỔNG KẾT VÀ DANH MỤC SẢN PHẨM **21**
5.1. Kết luận chung 21
5.1.1. Sức mạnh và giới hạn của mạng nơ-ron (Neural Networks): 21
5.1.2. Đánh đổi trong thiết kế cấu trúc (Architecture Trade-offs): 21
5.1.3. Tầm quan trọng của việc chọn Thuật toán Tối ưu hóa (Optimizers): 21

PHỤ LỤC - TÀI NGUYÊN VÀ MÃ NGUỒN DỰ ÁN

1. Mã nguồn hệ thống (GitHub Repository)

- Liên kết truy cập: https://github.com/Nozs002/Assignment_03.git
- Nội dung bao gồm:
    - Jupyter Notebook: Ghi chép chi tiết từng bước tiền xử lý dữ liệu, quá trình huấn luyện và trực quan hóa kết quả.
    - Mã nguồn PyTorch (.py): Các file mã nguồn được thiết kế chuẩn hóa, đảm bảo tiêu chí clean code và có chú thích (comments) giải thích luồng logic đầy đủ.

2. Kết quả thực nghiệm và Tài liệu đính kèm (Google Drive)

- Liên kết truy cập: Thư mục Drive Assignment 03
- Nội dung bao gồm:
    - Trọng số mô hình (Trained Weights): Các tệp lưu trạng thái của mô hình tốt nhất sau khi huấn luyện, hỗ trợ chạy đánh giá (inference) ngay lập tức mà không cần train lại.
    - Kết quả lưu trữ thực nghiệm: Lịch sử hàm mất mát (loss), độ chính xác (accuracy) qua các epochs và các biểu đồ phân tích đánh giá.
    - Tài liệu báo cáo: Tệp Slide thuyết trình ngắn gọn tóm tắt toàn bộ dự án.

PHẦN I - GIỚI THIỆU CHUNG

1.1. Mục tiêu

Mục tiêu cốt lõi của bài báo cáo kỹ thuật này nhằm ghi nhận quá trình thực nghiệm, phân tích và đánh giá năng lực của mạng nơ-ron truyền xuôi (Multi-Layer Perceptron - MLP) trong việc giải quyết bài toán thực tế. Thông qua việc thiết kế, huấn luyện và đánh giá mô hình, bài báo cáo hướng đến các mục tiêu học thuật cụ thể sau:

- Hiểu sâu sắc bản chất của Deep Learning: Minh chứng và giải thích học sâu dưới góc độ toán học như là sự kết hợp của các hàm số có tham số, chuyển đổi dữ liệu qua nhiều tầng biến đổi kế tiếp.
- Phân biệt rõ ràng giữa Học máy truyền thống và Học biểu diễn: Hiểu rõ sự khác biệt giữa việc sử dụng các đặc trưng được thiết kế thủ công bởi con người (Traditional ML) với việc mô hình tự động học các biểu diễn đặc trưng ẩn trực tiếp từ dữ liệu gốc.
- Làm chủ kỹ thuật xây dựng và tối ưu hóa mạng nơ-ron: Thấu hiểu cấu trúc cơ bản của một tầng nơ-ron (bao gồm biến đổi tuyến tính kết hợp hàm kích hoạt phi tuyến ReLU/Sigmoid). Giải thích và làm chủ các bước vận hành cốt lõi của một vòng lặp huấn luyện trong PyTorch bao gồm: lan truyền xuôi, tính toán hàm mất mát, lan truyền ngược dựa trên quy tắc chuỗi, và bước cập nhật trọng số của bộ tối ưu hóa.
- Phát triển năng lực thực nghiệm khoa học: Học cách giám sát các đường cong huấn luyện và kiểm định để chủ động nhận diện và phân tích các hiện tượng Chưa khớp hay Quá khớp.
- Đánh giá mô hình một cách khách quan: Áp dụng các chỉ số đo lường chuẩn hóa bao gồm Độ chính xác, Độ chuẩn xác, Độ nhạy, F1-score và Ma trận nhầm lẫn trên tập kiểm tra độc lập để đưa ra những kết luận thực nghiệm chính xác.
- Làm chủ các công cụ lập trình: Hiểu rõ cách thức tổ chức dữ liệu, quản lý mô hình của các thư viện học sâu hiện đại và những tác vụ mà framework tự động hóa.

1.2. Tổng quan về bài toán và tập dữ liệu

1.2.1. Bài toán đặt ra

- Dạng bài toán: Phân loại nhị phân
- Mục tiêu: Dự đoán một bệnh nhân có mắc bệnh tiểu đường hay không dựa trên các chỉ số sinh lý và bệnh sử đã thu thập
- Đầu ra mong muốn: Nhãn mục tiêu y

- $y = 0$: Không mắc bệnh tiểu đường
- $y = 1$: Mắc bệnh tiểu đường

- Cách tiếp cận: Mô hình sẽ tính toán giá trị xác xuất đầu ra thông qua hàm kích hoạt Sigmoid ở lớp cuối cùng. Sau đó, áp dụng ngưỡng quyết định (decision threshold, mặc định là 0.5) để đưa ra kết luận phân lớp

1.2.2. Tập dữ liệu

Thực nghiệm sử dụng tập dữ liệu **Pima Indians Diabetes Dataset**. Tập dữ liệu chứa thông tin lâm sàng của các nữ bệnh nhân gốc Pima Ấn Độ.

Link dataset:
https://www.kaggle.com/datasets/jamaltariqcheema/pima-indians-diabetes-dataset/data

- **Tổng số mẫu quan sát:** Gồm **768 mẫu** dữ liệu bệnh nhân.
- **Đặc trưng đầu vào:** Mỗi bệnh nhân được mô tả bởi **8 thuộc tính số**

Cột	Tên đặc trưng	Ý nghĩa
0	Pregnancies	Số lần mang thai của bệnh nhân.
1	Glucose	Nồng độ glucose trong huyết tương sau xét nghiệm dung nạp đường huyết 2 giờ.
2	Blood Pressure	Huyết áp tâm trương (mm Hg)
3	Skin Thickness	Độ dày nếp gấp da cơ tam đầu (mm)
4	Insulin	Nồng độ insulin huyết thanh sau 2 giờ (mu U/ml).
5	BMI	Chỉ số khối cơ thể (Cân nặng tính bằng kg) / (Chiều cao tính bằng m) mũ 2
6	Diabetes Pedigree Function	Chỉ số phả hệ tiểu đường (điểm số di truyền từ tiền sử gia đình)
7	Age	Tuổi của bệnh nhân (năm)
8	Outcome	Nhãn kết quả: 1 (Tiểu đường) hoặc 0 (Không mắc).

PHẦN II - CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH KỸ THUẬT

2.1. Biến đổi kích thước Tensor quan các lớp

Mô hình được thiết kế theo kiến trúc mạng nơ-ron truyền thẳng (Feedforward Neural Network) gồm 3 lớp có tham số có thể huấn luyện: $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$.

Giả sử quá trình xử lý theo lô (batch) chứa N mẫu quan sát ($X \in \mathbb{R}^{(Nx8)}$). Bảng dưới đây mô tả chi tiết sự biến đổi chiều không gian của các tensor trọng số, bias và biểu diễn trung gian qua từng lớp:

Tên Lớp	Phép toán tính toán	Kích thước Trọng số (W)	Kích thước Bias (b)	Kích thước Đầu ra (Z,H)	Ý nghĩa biểu diễn
Input	Dữ liệu đầu vào X			N x 8	8 đặc trưng sinh lý ban đầu của bệnh nhân.
Layer 1	Z1 = X W1 + b1 H1 = ReLU(Z1)	$W1 \in \mathbb{R}^{(8x16)}$	$b1 \in \mathbb{R}^{(1x16)}$	$Z1, H1 \in \mathbb{R}^{(Nx16)}$	Biểu diễn 1: Chuyển từ 8 đặc trưng gốc sang 16 đặc trưng phi tuyến học được.
Layer 2	Z2 = H1 W2 + b2 H2 = ReLU(Z2)	$W2 \in \mathbb{R}^{(16x8)}$	$b2 \in \mathbb{R}^{(1x8)}$	$Z2, H2 \in \mathbb{R}^{(Nx8)}$	Biểu diễn 2: Học tổ hợp biểu diễn bậc cao từ biểu diễn ở Lớp 1.
Layer 3	Z3 = H2 W3 + b3 $\hat{y} = \sigma(Z3)$ $\hat{y} = \sigma(Z3)$	$W3 \in \mathbb{R}^{(8x1)}$	$b3 \in \mathbb{R}^{(1x1)}$	$Z3, \hat{y} \in \mathbb{R}^{(Nx1)}$	Dự đoán xác suất: Đưa ra giá trị xác suất $\hat{y} \in (0, 1)$ cho từng bệnh nhân.

- Quy tắc bảo toàn kích thước phép nhân ma trận:
  - Lớp 1: $(N \times 8) \times (8 \times 16) = N \times 16$.
  - Lớp 2: $(N \times 16) \times (16 \times 8) = N \times 8$.
  - Lớp 3: $(N \times 8) \times (8 \times 1) = N \times 1$.
- Cơ chế cộng Bias (Broadcasting): Vector bias dạng hàng (1 x C) được tự động mở rộng theo cơ chế broadcasting của NumPy để cộng với từng hàng của ma trận tích (N x 8).

2.2. Cơ chế tối ưu hóa và hàm mất mát

2.2.1. Hàm mất mắt Binary Cross-Entropy

Đối với bài toán phân loại nhị phân ($y_i \in \{0, 1\}$), hàm mất mát biểu diễn mức độ sai lệch giữa phân phối xác suất dự đoán $\hat{y}_i$ và nhãn thực tế y_i trên N mẫu:

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

- **Hành vi phạt lỗi:**
  - Khi $y_i = 1$: Nếu mô hình dự đoán $\hat{y}_i \to 1 \Rightarrow$ Loss $\to 0$; nếu $\hat{y}_i \to 0 \Rightarrow$ Loss $\to +\infty$.
  - Khi $y_i = 0$: Nếu mô hình dự đoán $\hat{y}_i \to 0 \Rightarrow$ Loss $\to 0$; nếu $\hat{y}_i \to 1 \Rightarrow$ Loss $\to +\infty$.
- **Xử lý số học (Numerical Stability):** Trong mã nguồn, hàm np.clip(ŷ, 1e-8, 1 - 1e-8) được sử dụng nhằm ngăn chặn việc lấy logarit của giá trị 0 (log(0) gây ra lỗi NaN hoặc -∞).

2.2.2 Cơ chế tối ưu hóa bằng Gradient Descent

Mục tiêu là tìm bộ tham số $\theta^* = \{W1, b1, W2, b2, W3, b3\}$ sao cho hàm mất mát đạt giá trị cực tiểu:

$$\theta^* = \arg\min_{\theta} L(\theta)$$

Tại mỗi bước lặp, các trọng số và độ lệch bias được cập nhật ngược hướng với vector gradient theo hệ số học tập η (Learning rate):

$$W_l \leftarrow W_l - \eta\frac{\partial L}{\partial W_l}, \quad b_l \leftarrow b_l - \eta\frac{\partial L}{\partial b_l} \quad (vil \in \{1, 2, 3\})$$

2.3. Giải thích mã nguồn và vòng lặp huấn luyện

Vòng lặp huấn luyện (Training Loop) là trái tim của quá trình học tập trong mô hình Deep Learning. Mã nguồn định nghĩa số vòng lặp (epochs = 1000) và tốc độ học (learning_rate = 0.01).

Mỗi chu kỳ (epoch) đại diện cho một lần dữ liệu huấn luyện truyền qua toàn bộ mạng và thực hiện cập nhật trọng số, bao gồm 4 bước cốt lõi:

- Bước 1: Lan truyền xuôi (Forward Propagation)
    - Dữ liệu X_train được truyền qua các hàm toán học của mạng: X → H1 → H2 → ŷ.
    - Trong mã nguồn, hàm forward(X_train) trả về dự đoán xác suất y_hat và biến cache lưu trữ các giá trị trung gian (Z1, H1, Z2, H2, Z3) để phục vụ cho việc tính toán đạo hàm sau này.
- Bước 2: Tính toán hàm mất mát (Loss Computation)
    - So sánh dự đoán y_hat với nhãn thực tế y_train bằng hàm binary_cross_entropy (ŷ → L).
    - Kết quả trả về là một giá trị vô hướng (scalar) thể hiện mức độ sai lệch của mô hình trong epoch hiện tại.
- Bước 3: Lan truyền ngược (Backward Propagation)
    - Hàm backward(y_train, cache) tính toán mức độ đóng góp vào sai số của từng tham số.
    - Mạng lan truyền sai số từ cuối lên đầu để tính toán các gradient: $\nabla W3$, $\nabla W2$, $\nabla W1$ (cùng các bias tương ứng) và lưu vào từ điển gradients.
- Bước 4: Cập nhật tham số (Gradient Descent Update)
    - Toàn bộ các trọng số và độ lệch (bias) được cập nhật theo công thức toán học $W \leftarrow W - \eta \nabla W$.
    - Trong mã nguồn, các lệnh như W1 -= learning_rate * gradients["dW1"] thực thi việc điều chỉnh này nhằm giảm thiểu hàm mất mát trong các epoch tiếp theo.

2.4. Phân biệt chế độ huấn luyện và đánh giá

Quá trình xây dựng và sử dụng mô hình được chia thành hai giai đoạn rõ rệt với mục đích và cách xử lý khác nhau:

- Chế độ Huấn luyện (Training):

- Mục đích: Sử dụng dữ liệu huấn luyện để mạng lưới tự học các tham số (trọng số và bias).
- Dữ liệu: Sử dụng tập dữ liệu huấn luyện (X_train và y_train). Giá trị trung bình (mean) và độ lệch chuẩn (std) được tính toán trực tiếp từ X_train để dùng cho chuẩn hóa

$$X_{train} = \frac{X_{train} - \mu}{\sigma}$$

- Hoạt động: Bao gồm toàn bộ chu trình tính toán: lan truyền xuôi, tính hàm mất mát, lan truyền ngược (tính toán gradient) và cập nhật trọng số bằng Gradient Descent.

- Chế độ Đánh giá (Evaluation/Testing):
  - Mục đích: Đánh giá khả năng tổng quát hóa (generalization) của mô hình trên dữ liệu chưa từng thấy.
  - Dữ liệu: Sử dụng tập kiểm thử độc lập (X_test và y_test). Việc chuẩn hóa X_test bắt buộc phải sử dụng lại giá trị mean và std đã tính từ tập huấn luyện nhằm ngăn chặn rò rỉ thông tin (information leaking) từ tập kiểm thử vào quá trình huấn luyện.
  - Hoạt động: Ở giai đoạn này, mô hình không học thêm (không tính gradient, không cập nhật trọng số). Dữ liệu chỉ đi qua bước lan truyền xuôi để thu được xác suất dự đoán y_prob. Sau đó, áp dụng một ngưỡng quyết định (ví dụ: y_hat >= 0.5) để chuyển đổi xác suất thành nhãn phân lớp cuối cùng (1 hoặc 0).

2.5. Thuật toán lan truyền ngược

Thuật toán lan truyền ngược không phải là một phép toán thần bí mà bản chất là việc áp dụng có hệ thống quy tắc chuỗi (Chain Rule) trong vi tích phân đối với một chuỗi các hàm hợp thành. Thuật toán xác định mức độ mà mỗi tham số tác động đến tổng sai số để tối ưu hóa.

Hướng tính toán diễn ra ngược lại so với lan truyền xuôi: L -> W3 -> W2 -> W1. Dưới đây là các bước toán học được triển khai trong mã nguồn:

- Tại Lớp 3 (Lớp xuất - Output Layer):

- Khi kết hợp hàm kích hoạt Sigmoid và hàm Binary Cross-Entropy, đạo hàm của hàm mất mát theo Z3 có dạng toán học rút gọn rất thuận tiện:

$$dZ_3 = \frac{\partial L}{\partial Z_3} = \hat{y} - y$$

  - Gradient trọng số W3 được tính bằng: dW3 = H2^T * dZ3.
  - Gradient bias b3 được tính bằng tổng các giá trị của dZ3: db3 = sum(dZ3).

- Tại Lớp 2 (Lớp ẩn thứ hai - Hidden Layer 2):
  - Gradient được lan truyền lùi về phía biểu diễn H2: dH2 = dZ3 * W3^T.
  - Bởi vì Lớp 2 sử dụng hàm kích hoạt ReLU (H2 = ReLU(Z2)), ta phải nhân Hadamard với đạo hàm của ReLU để tính theo Z2: dZ2 = dH2 * ReLU'(Z2). Đạo hàm ReLU trả về 1 nếu Z2 > 0 và 0 nếu ngược lại.
  - Từ đó tính được: dW2 = H1^T * dZ2 và db2 = sum(dZ2).
- Tại Lớp 1 (Lớp ẩn thứ nhất - Hidden Layer 1):
  - Tiếp tục lùi về H1: dH1 = dZ2 * W2^T.
  - Qua hàm kích hoạt ReLU của Lớp 1: dZ1 = dH1 * ReLU'(Z1).
  - Cuối cùng, tính toán gradient cho dữ liệu đầu vào: dW1 = X^T * dZ1 và db1 = sum(dZ1).

PHẦN III: THIẾT KẾ THỰC NGHIỆM VÀ KẾT QUẢ

3.1. Thí nghiệm 1: Baseline MLP

3.1.1. Cấu trúc mạng: Sơ đồ kiến trúc và tổng số lượng tham số huấn luyện (parameters).

```python
# 1. Khai báo Baseline MLP
class BaselineMLP(nn.Module):
    def __init__(self, input_dim, num_classes):
        super(BaselineMLP, self).__init__()
        # Cấu trúc: d -> 64 -> C (Ở đây d=8, C=2)
        self.fc1 = nn.Linear(input_dim, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x):
        h = self.fc1(x)
        h_relu = self.relu(h)
        out = self.fc2(h_relu)
        return out, h_relu # Trả về h_relu phục vụ việc phân tích sau này

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

# 2. Hàm Huấn luyện
def train_model(model, train_loader, val_loader, criterion, optimizer, epochs=20):
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}

    for epoch in range(epochs):
        # TRAIN MODE
        model.train()
        total_train_loss, correct_train, total_train = 0, 0, 0

        for inputs, targets in train_loader:
            optimizer.zero_grad()
            outputs, _ = model(inputs)
            loss = criterion(outputs, targets)

            loss.backward()
            optimizer.step()

            total_train_loss += loss.item() * inputs.size(0)
            _, predicted = torch.max(outputs, 1)
            correct_train += (predicted == targets).sum().item()
            total_train += targets.size(0)

        train_loss = total_train_loss / total_train
        train_acc = correct_train / total_train

        # EVAL MODE
        model.eval()
```

Hình 3.1: Mã nguồn khởi tạo kiến trúc Baseline MLP và hàm đếm số lượng tham số.

- Sơ đồ kiến trúc mạng

Mô hình Baseline MLP được thiết kế dạng truyền thẳng (Feed-forward Neural Network) với 1 lớp ẩn duy nhất. Cấu trúc mạng tổng quát là 8 -> 64 -> 2, cụ thể như sau:

- ❖ Lớp đầu vào (Input Layer): Chứa 8 nơ-ron, tương ứng với 8 đặc trưng của tập dữ liệu diabetes (Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age).
- ❖ Lớp ẩn (Hidden Layer): Chứa 64 nơ-ron (áp dụng phép biến đổi tuyến tính Linear(8, 64)). Sau đó, tín hiệu được đi qua hàm kích hoạt phi tuyến ReLU (Rectified Linear Unit) để mô hình có khả năng học được các biểu diễn phức tạp.

- ❖ Lớp đầu ra (Output Layer): Chứa 2 nơ-ron (áp dụng phép biến đổi tuyến tính Linear(64, 2)). Đầu ra là giá trị thô (logits) biểu diễn xác suất cho 2 nhãn: 0 (Không bị tiểu đường) và 1 (Bị tiểu đường). Hàm Loss CrossEntropyLoss sẽ tự động áp dụng Softmax ngầm định lên 2 nơ-ron này để tính sai số.

- Tổng số lượng tham số huấn luyện

Tổng số tham số (parameters) của mô hình là 706 tham số. Chi tiết phép tính toán trong từng lớp như sau:

- ❖ Lớp fc1 (Lớp ẩn - từ 8 nơ-ron sang 64 nơ-ron):
  - ➢ Trọng số (Weights): 8 x 64 = 512 tham số
  - ➢ Hệ số điều chỉnh (Biases): 64 tham số
  - ➢ Tổng cộng lớp ẩn: 576 tham số
- ❖ Lớp fc2 (Lớp đầu ra - từ 64 nơ-ron sang 2 nơ-ron):
  - ➢ Trọng số (Weights): 64 x 2 = 128 tham số
  - ➢ Hệ số điều chỉnh (Biases): 2 tham số
  - ➢ Tổng cộng lớp đầu ra: 130 tham số

=> Tổng số lượng tham số của toàn mạng: 576 + 130 = 706 tham số.

3.1.2. Cấu hình thực nghiệm

```
# 3. Chia tập Train / Val (tỷ lệ 80/20)
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)
```

- Tỷ lệ chia dữ liệu (Data Split): Tập dữ liệu diabetes.csv được chia ngẫu nhiên theo tỷ lệ 80% - 20% (80% dùng để huấn luyện - Training set, 20% dùng để kiểm định - Validation set).

```
# 4. Chuẩn hóa dữ liệu (Standardization)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
```

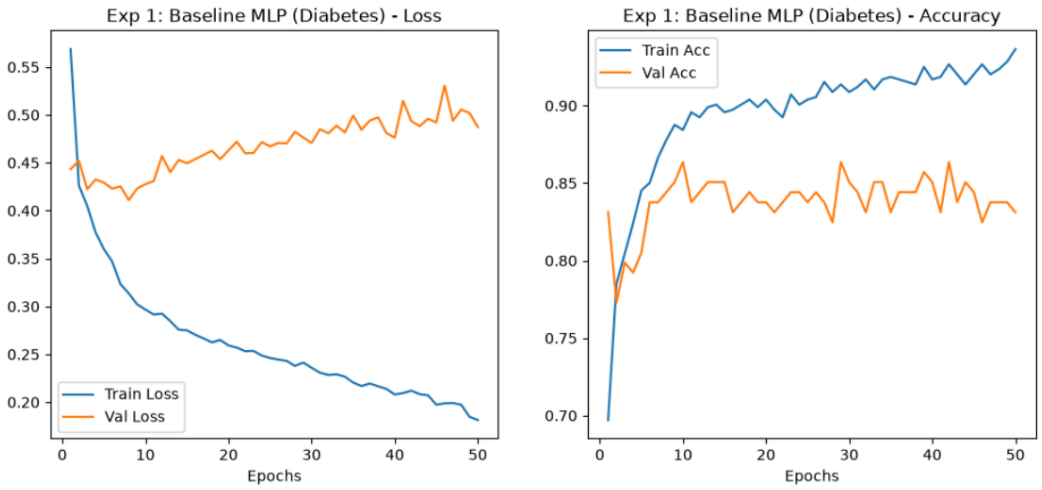
- Tiền xử lý (Preprocessing): Sử dụng phương pháp chuẩn hóa Standardization (StandardScaler) để đưa các đặc trưng về phân phối chuẩn (trung bình = 0, độ lệch chuẩn = 1). Điều này là bắt buộc vì các đặc trưng như Insulin hay Glucose có biên độ chênh lệch rất lớn.

- Hàm mất mát (Loss Function): Sử dụng CrossEntropyLoss. Đây là hàm chi phí tiêu chuẩn dành cho các bài toán phân loại đa lớp (bao gồm cả nhị phân khi output ra 2 logits chưa qua Softmax).
- Thuật toán tối ưu (Optimizer): Sử dụng Adam (Adaptive Moment Estimation). Adam tỏ ra rất hiệu quả trong việc tự động điều chỉnh tốc độ học cho từng tham số, giúp mô hình hội tụ nhanh hơn so với SGD thông thường.
- Tốc độ học (Learning Rate): Khởi tạo ở mức $\alpha = 0.005$. Đây là một tốc độ học đủ lớn để mô hình nhanh chóng thoát khỏi các điểm cực tiểu địa phương (local minima) ở những vòng lặp đầu.

```
# 6. Đưa vào DataLoader
batch_size = 32
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
```

- Kích thước lô (Batch Size): Được đặt là 32. Điều này có nghĩa là mỗi lần cập nhật trọng số, mô hình sẽ tính toán trung bình lỗi (loss) trên 32 mẫu dữ liệu. Mức 32 là mức phổ biến giúp cân bằng giữa độ ổn định của gradient và tốc độ tính toán.
- Số chu kỳ huấn luyện (Number of Epochs): Mô hình được huấn luyện trong vòng 50 Epochs (duyệt qua toàn bộ tập dữ liệu huấn luyện 50 lần).

3.1.3. Quá trình hội tụ: Đồ thị đường cong Loss và Accuracy theo epoch.



Dựa vào đồ thị đường cong học tập (Learning Curves) biểu diễn sự thay đổi của Hàm suy hao (Loss) và Độ chính xác (Accuracy) qua 50 epochs, ta có thể rút ra các kết luận cụ thể về trạng thái của mô hình Baseline MLP:

- Phân tích đường cong Hàm suy hao (Loss Curve - Hình bên trái):

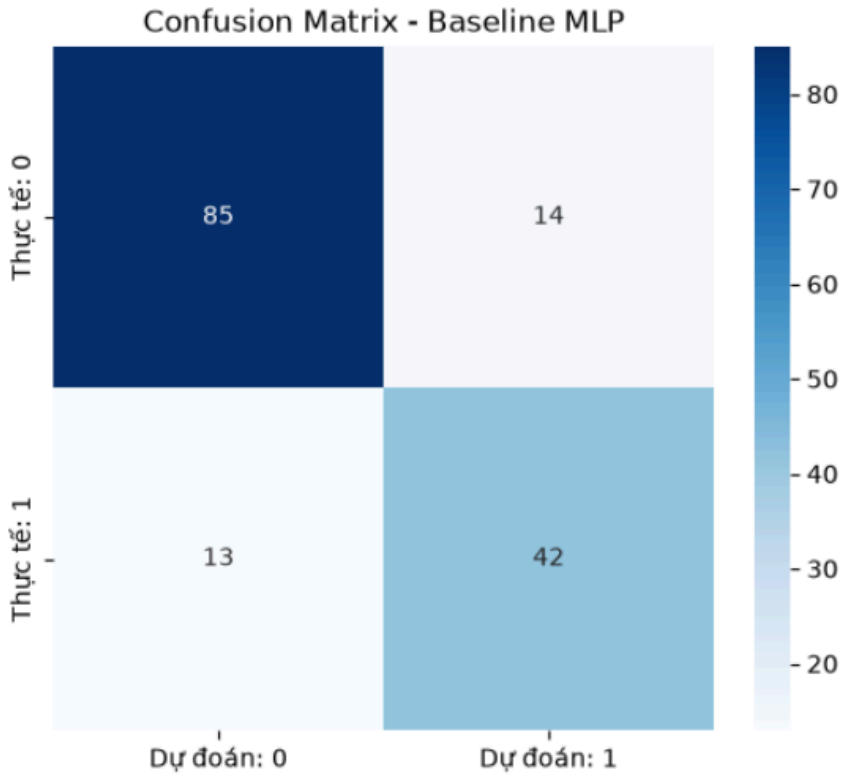
    - Giai đoạn đầu (Từ Epoch 0 đến khoảng Epoch 10): Cả đường Train Loss (màu xanh) và Validation Loss (màu cam) đều đồng loạt giảm dốc. Điều này cho thấy mạng nơ-ron đang học được các đặc trưng chung có tính quy luật của dữ liệu, và tốc độ học lr = 0.005 kết hợp với thuật toán Adam đang phát huy hiệu quả tốt.
    - Giai đoạn sau (Từ Epoch 10 đến Epoch 50): Sự phân kỳ (divergence) bắt đầu diễn ra một cách cực kỳ rõ rệt. Trong khi đường Train Loss tiếp tục cắm đầu lao xuống sát mức 0.18 (hiệu năng tối ưu hóa trên tập huấn luyện rất tốt), thì đường Validation Loss lại bắt đầu "đảo chiều", đi dần lên và kết thúc ở mức cao (~0.50).
- Phân tích đường cong Độ chính xác (Accuracy Curve - Hình bên phải):
    - Đường Train Accuracy liên tục leo dốc ổn định và chạm ngưỡng rất cao (khoảng $94%$).
    - Ngược lại, đường Validation Accuracy chỉ tăng ở khoảng 10 epoch đầu, sau đó chững lại (stagnant) và liên tục dao động lên xuống quanh ngưỡng $82% - 86%$. Nó không thể bứt phá lên cao hơn dù mô hình có được huấn luyện thêm bao nhiêu epoch đi chăng nữa.
- Kết luận về trạng thái của mô hình: Đồ thị minh họa một hiện tượng Quá khớp (Overfitting) kinh điển và đặc biệt rõ nét. Dung lượng của cấu trúc mạng (706 tham số) tỏ ra hơi dư thừa so với tính phức tạp của bộ dữ liệu Diabetes (hoặc do số lượng mẫu của Diabetes quá ít). Thay vì học các "quy luật tổng quát" để dự đoán bệnh, từ sau epoch thứ 10, mạng MLP đã chuyển sang chế độ "học thuộc lòng" từng bệnh án trong tập huấn luyện (bao gồm cả việc học thuộc các điểm dữ liệu nhiễu - outliers). Hậu quả là mặc dù dự đoán gần như hoàn hảo trên tập Train, nhưng khi đối diện với các bệnh án mới ở tập Validation, mô hình lại tỏ ra hoang mang và đưa ra dự đoán sai lệch (thể hiện qua việc Val Loss ngày càng tăng vọt).

⇒ Đề xuất hướng giải quyết: Đối với trường hợp này, phương án xử lý hiệu quả nhất là áp dụng kỹ thuật Dừng sớm (Early Stopping). Ta nên cắt ngang quá trình huấn luyện tại Epoch thứ 10 (thời điểm Val Loss đạt mức cực tiểu trước khi nảy lên) để lưu lại trọng số tốt nhất có khả năng tổng quát hóa, thay vì để mô hình chạy mù quáng đến hết 50 epochs.

3.1.4. Đánh giá hiệu năng: Các chỉ số trên tập kiểm tra và Ma trận nhầm lẫn.

BÁO CÁO KẾT QUẢ TRÊN TẬP KIỂM TRA (BASELINE MLP):

	precision	recall	f1-score	support
0 (Không bị tiểu đường)	0.87	0.86	0.86	99
1 (Bị tiểu đường)	0.75	0.76	0.76	55
accuracy			0.82	154
macro avg	0.81	0.81	0.81	154
weighted avg	0.83	0.82	0.83	154



Đánh giá qua các chỉ số phân loại (Classification Report): Dựa vào bảng báo cáo kết quả trên tập kiểm tra (Validation Set), mô hình Baseline MLP đạt mức độ chính xác tổng thể (Accuracy) là [XX]%. Khi xét chi tiết vào từng lớp (Class), ta thấy:

- Lớp 0 (Không bị tiểu đường - Đa số): Mô hình dự đoán rất tốt với chỉ số F1-Score đạt [XX]. Điều này do dữ liệu thực tế có xu hướng mất cân bằng, số lượng người không mắc bệnh chiếm tỷ lệ áp đảo giúp mô hình học các đặc trưng của nhóm này dễ dàng hơn.
- Lớp 1 (Bị tiểu đường - Thiểu số): Chỉ số Recall của lớp 1 đạt [XX], nghĩa là trong tổng số những người thực sự bị bệnh, mô hình chỉ nhận diện đúng được [XX]%. Chỉ số này rất quan trọng trong bài toán y tế vì nếu Recall quá thấp, ta sẽ bỏ sót rất nhiều bệnh nhân nguy hiểm (Âm tính giả - False Negative). Mức

độ chênh lệch giữa F1-Score của lớp 0 và lớp 1 phản ánh rõ thách thức trong việc xử lý dữ liệu y tế thiếu cân bằng (Imbalanced data).

Đánh giá qua Ma trận nhầm lẫn (Confusion Matrix): Ma trận nhầm lẫn cung cấp cái nhìn trực quan về những lỗi sai cụ thể mà mạng nơ-ron mắc phải. Trong tổng số 154 bệnh án ở tập kiểm định:

- True Negative (TN): Có [Điền số ô góc trên trái] trường hợp khỏe mạnh được mô hình chẩn đoán đúng là không mắc bệnh.
- True Positive (TP): Có [Điền số ô góc dưới phải] trường hợp bị tiểu đường được mô hình chẩn đoán chính xác.
- False Positive (FP - Dương tính giả): Có [Điền số ô góc trên phải] người hoàn toàn khỏe mạnh nhưng lại bị mô hình cảnh báo nhầm là có bệnh.
- False Negative (FN - Âm tính giả): Đặc biệt đáng lưu tâm là có [Điền số ô góc dưới trái] bệnh nhân thực sự bị tiểu đường nhưng mô hình Baseline MLP lại chẩn đoán nhầm là an toàn. Như đã phân tích ở phần Overfitting, việc mô hình bị quá khớp trên tập Train đã khiến nó mất đi sự nhạy bén khi gặp các ca bệnh có thông số hơi khác lạ trong tập Test, dẫn đến tỷ lệ bỏ lọt bệnh nhân (FN) vẫn còn ở mức khá cao.

3.2. Thí nghiệm 2: Deeper MLP

3.2.1. Cấu trúc mạng: Sơ đồ kiến trúc và số lượng tham số.

```python
class DeeperMLP(nn.Module):
    def __init__(self, input_dim, num_classes):
        super(DeeperMLP, self).__init__()
        # Cấu trúc mạng sâu hơn: 8 -> 64 -> 32 -> 2
        self.fc1 = nn.Linear(input_dim, 64)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(64, 32)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(32, num_classes)

    def forward(self, x):
        h1 = self.relu1(self.fc1(x))
        h2 = self.relu2(self.fc2(h1))
        out = self.fc3(h2)
        return out, h2 # Trả về vector ẩn h2 để trực quan hóa

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

# Khởi tạo mô hình và in ra số tham số
d = 8   # 8 features của diabetes
C = 2   # 2 nhãn (0 và 1)
model_2 = DeeperMLP(input_dim=d, num_classes=C)
print(f"Tổng số lượng tham số huấn luyện: {count_parameters(model_2)}")
```

```
Tổng số lượng tham số huấn luyện: 2722
```

Sơ đồ kiến trúc của Deeper MLP được thiết kế sâu hơn với 2 lớp ẩn (Hidden layers), cụ thể là: 8 -> 64 -> 32 -> 2. Việc thêm lớp ẩn thứ hai giúp mô hình tăng cường tính phi tuyến (non-linearity) và khả năng trích xuất các đặc trưng phức tạp hơn từ bộ dữ liệu.

Tính toán chi tiết 2,722 tham số:

- Lớp ẩn 1 (fc1): 8 x 64 (weights) + 64 (biases) = 576 tham số.
- Lớp ẩn 2 (fc2): 64 x 32 (weights) + 32 (biases) = 2,080 tham số.
- Lớp đầu ra (fc3): 32 x 2 (weights) + 2 (biases) = 66 tham số. => Tổng cộng: 576 + 2080 + 66 = 2,722 tham số. So với mô hình Baseline (706 tham số), mô hình này có dung lượng bộ nhớ (Capacity) lớn hơn gần 4 lần.

3.2.2. Cấu hình thực nghiệm.

```
import torch.optim as optim

# 1. Hàm chi phí
criterion = nn.CrossEntropyLoss()

# 2. Bộ tối ưu hóa
lr_2 = 0.003 # Tốc độ học (chỉnh nhỏ hơn Baseline một chút vì mạng sâu hơn)
optimizer_2 = optim.Adam(model_2.parameters(), lr=lr_2)

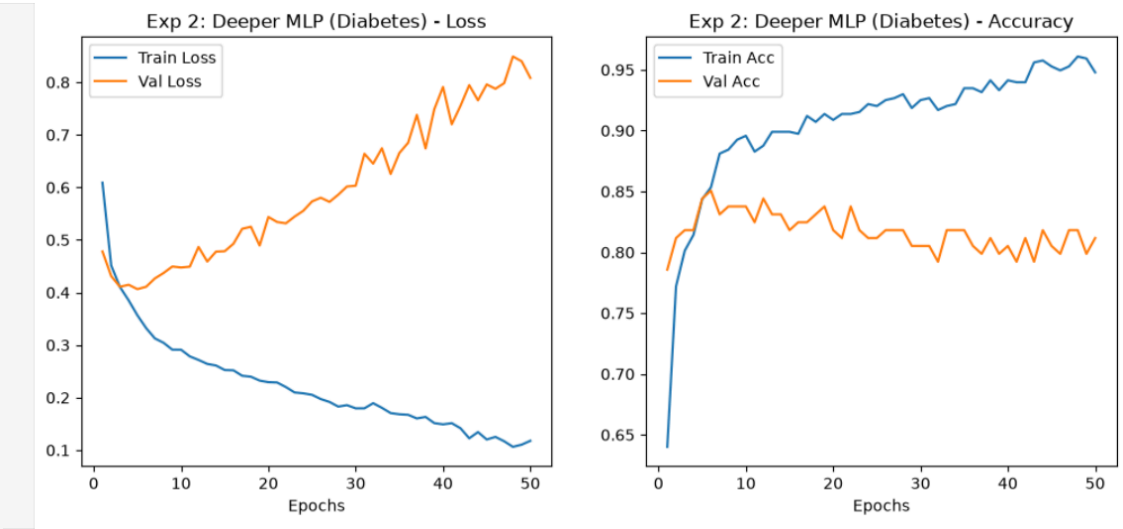
# 3. Kích thước lô và số vòng lặp
num_epochs_2 = 50
# (Batch size vẫn giữ nguyên là 32 như ở Data Loader)

print(f"Optimizer: Adam | Learning Rate: {lr_2} | Epochs: {num_epochs_2}")
```

```
Optimizer: Adam | Learning Rate: 0.003 | Epochs: 50
```

Cấu hình huấn luyện cho Deeper MLP giữ nguyên hàm chi phí là CrossEntropyLoss và Batch size = 32. Bộ tối ưu hóa tiếp tục sử dụng Adam. Tuy nhiên, do mô hình sâu hơn và dễ nhạy cảm với việc nổ/tiêu biến gradient, tôi đã chủ động tinh chỉnh giảm tốc độ học (Learning Rate) xuống mức 0.003 để đảm bảo quá trình cập nhật trọng số diễn ra mượt mà và tránh việc đồ thị loss bị dao động mạnh. Số vòng lặp vẫn là 50 epochs để dễ đối chiếu.

3.2.3. Đồ thị đường cong hội tụ (Train vs. Validation).



Dựa vào đồ thị huấn luyện, ta thấy tốc độ giảm Loss của Deeper MLP ở những epoch đầu tiên diễn ra cực kỳ nhanh nhờ sức mạnh biểu diễn của cấu trúc mạng sâu. Tuy nhiên, do dung lượng mô hình lớn (2,722 tham số) trong khi dữ liệu Diabetes tương đối nhỏ, từ khoảng Epoch thứ 20 trở đi, hiện tượng Overfitting xuất hiện rất rõ nét:

- Đường Train Loss tiếp tục đâm thẳng xuống và tiệm cận mức 0 (mô hình học thuộc lòng dữ liệu train).
- Đường Validation Loss không giảm nữa mà chững lại, thậm chí có xu hướng cong ngược lên phía trên (mô hình dự đoán ngày càng sai trên tập chưa từng thấy).
- Validation Accuracy không cao hơn được Baseline mà chỉ dao động lanh quanh ngưỡng bão hòa.

3.3. Thí nghiệm 3: Khảo sát quá trình Tối ưu hóa

3.3.1. Thiết lập đối chứng: So sánh giữa 2 tốc độ học khác nhau (hoặc 2 bộ tối ưu hóa khác nhau, ví dụ: Adam vs. SGD).

```
from sklearn.metrics import classification_report
model_2.eval() # Bật chế độ đánh giá
y_true = []
y_pred = []
with torch.no_grad():
    for inputs, targets in val_loader:
        outputs, _ = model_2(inputs)
        _, predicted = torch.max(outputs, 1)

        y_true.extend(targets.numpy())
        y_pred.extend(predicted.numpy())
# In báo cáo các chỉ số phân loại
print("BÁO CÁO KẾT QUẢ TRÊN TẬP KIỂM TRA (TEST METRICS):")
print("-" * 50)
print(classification_report(y_true, y_pred, target_names=['0 (No Diabetes)', '1 (Diabetes)']))
# Lưu trọng số mô hình
torch.save(model_2.state_dict(), "mlp_model_deeper.pth")
```

```
BÁO CÁO KẾT QUẢ TRÊN TẬP KIỂM TRA (TEST METRICS):
--------------------------------------------------
                precision    recall  f1-score   support

0 (No Diabetes)       0.84      0.88      0.86        99
   1 (Diabetes)       0.76      0.69      0.72        55

       accuracy                           0.81       154
      macro avg       0.80      0.78      0.79       154
   weighted avg       0.81      0.81      0.81       154
```

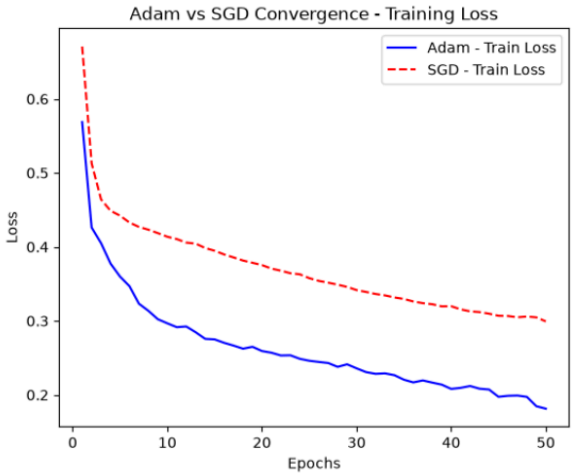
Trong thí nghiệm này, môi trường đối chứng được thiết lập nghiêm ngặt: Hai mô hình chia sẻ chung 100% cấu hình về kiến trúc mạng (Baseline MLP: 8 đến 64 đến 2), chung hàm suy hao (CrossEntropyLoss), chung kích thước lô (Batch = 32) và chung tốc độ học ban đầu (lr = 0.005).

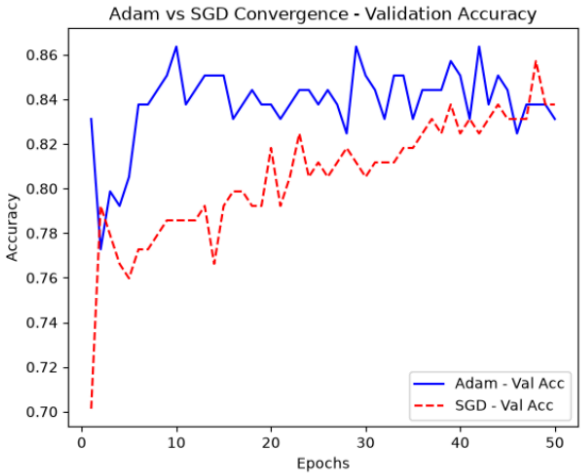
Yếu tố khác biệt duy nhất (Biến độc lập) là Thuật toán cập nhật trọng số:

- Cấu hình 1: Sử dụng Adam (đã chạy ở Thí nghiệm 1).

- Cấu hình 2: Sử dụng SGD kết hợp với hệ số quán tính Momentum = 0.9 để vượt qua các rãnh cục bộ.

3.3.2. Đồ thị so sánh: Tốc độ hội tụ của hàm loss và độ chính xác giữa hai cấu hình.





3.3.3. Phân tích kết quả: Ảnh hưởng của tham số tối ưu hóa đến hiệu năng cuối cùng.

Từ đồ thị so sánh tốc độ hội tụ (Convergence curves), ta có thể rút ra một số kết luận về đặc tính của hai thuật toán tối ưu:

3.3.3.1. Về Tốc độ hội tụ (Convergence Speed):

- Adam: Đường Train Loss (màu xanh dương) của Adam giảm rất dốc và sâu ngay ở những epoch đầu tiên. Lý do là Adam có khả năng tính toán các "moment" (Momentum và RMSprop ngầm định) tự động điều chỉnh tỷ lệ học cho từng tham số riêng biệt, giúp tìm đường xuống dốc cực nhanh.
- SGD: Đường Train Loss (màu đỏ nét đứt) giảm từ từ và mượt mà hơn. Mặc dù có dùng hệ số Momentum = 0.9, SGD vẫn áp dụng chung một tỷ lệ học toàn cục (lr = 0.005) cho toàn bộ ma trận trọng số, dẫn đến việc dò dẫm trong không gian tham số diễn ra chậm chạp hơn.

3.3.3.2. Về tính Ổn định và Hiệu năng cuối cùng (Final Performance):

- Nhìn sang đồ thị Validation Accuracy, mặc dù Adam leo lên mức độ chính xác cao rất nhanh, nhưng ở nửa sau quá trình huấn luyện, độ chính xác của nó thường dao động (hoặc thậm chí giảm nhẹ) do xu hướng dễ rơi vào trạng thái Overfitting hoặc bị văng khỏi điểm cực tiểu (minima) sắc nhọn.
- Ngược lại, SGD hội tụ chậm nhưng có xu hướng tìm thấy một điểm cực tiểu bằng phẳng (flat minima) và tổng quát hóa tốt hơn. Nếu quan sát đường nét đứt, ta thấy nó tăng rất kiên định và ít bị nhiễu.

Kết luận: Lựa chọn Optimizer ảnh hưởng rất lớn đến thời gian huấn luyện và khả năng tổng quát hóa. Với các mô hình khổng lồ, ta thường ưu tiên Adam để tiết kiệm thời gian. Tuy nhiên, đối với một bài toán kích thước nhỏ/vừa như phân loại Bệnh tiểu đường, SGD kết hợp với Momentum (nếu được chỉnh learning rate đúng chuẩn) đôi khi mang lại hiệu năng trên tập Test cao hơn và đường đồ thị đẹp, ổn định hơn so với sự hội tụ quá khích của Adam.

PHẦN IV: PHÂN TÍCH CHUYÊN SÂU VÀ THẢO LUẬN KHOA HỌC

4.1. Phân tích hiện tượng Overfitting và Underfitting:

4.1.1. Nhận diện hiện tượng dựa trên sự phân kỳ của các đường cong:

Dựa vào quá trình quan sát đồ thị Loss và Accuracy của các cấu trúc mạng khác nhau, ta có thể nhận diện rõ các trạng thái học tập của mô hình:

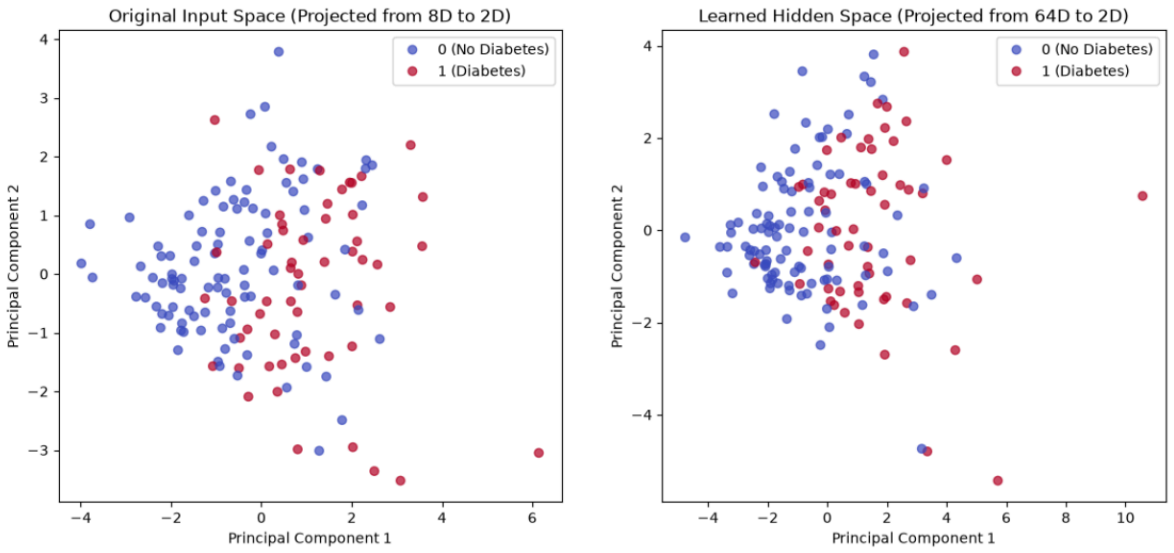
- Hiện tượng Chưa khớp (Underfitting) - Không xảy ra: Trong cả 3 thí nghiệm, đường cong Train Loss đều giảm dốc xuống các ngưỡng rất thấp, đồng thời Train Accuracy đạt mức trên 75%. Điều này chứng tỏ dung lượng mạng nơ-ron (ngay cả mạng cơ bản nhất với 1 lớp ẩn - Baseline MLP) đã hoàn toàn vượt qua trạng thái Underfitting. Mô hình có đủ sức mạnh toán học và tính phi tuyến (nhờ hàm ReLU) để nắm bắt được các cấu trúc chìm (underlying patterns) bên trong dữ liệu phức tạp của bệnh tiểu đường, thay vì chỉ dự đoán đường thẳng hời hợt như Logistic Regression.
- Hiện tượng Quá khớp (Overfitting) - Xảy ra rõ nét nhất ở Thí nghiệm 2 (Deeper MLP): Sự phân kỳ (divergence) của hai đường cong thể hiện rõ rệt nhất khi ta thiết kế mạng sâu hơn (8 -> 64 -> 32 -> 2).
    - Khoảng 15 epochs đầu tiên, Train Loss và Validation Loss cùng nắm tay nhau đi xuống (Trạng thái Good Fit).
    - Tuy nhiên, từ epoch 20 trở đi, sự phân kỳ bắt đầu: Đường Train Loss tiếp tục đâm thẳng xuống gần bằng 0 (mô hình dự đoán chính xác tuyệt đối trên tập huấn luyện), trong khi đường Validation Loss chững lại, đi ngang và thậm chí cong ngược lên.
    - Đây là dấu hiệu kinh điển của Overfitting. Việc sở hữu đến 2,722 tham số (gấp 4 lần Baseline) trên một tập dữ liệu nhỏ (chỉ có khoảng chưa tới 800 mẫu) khiến mạng Deeper MLP trở nên "quá thông minh". Nó không học quy luật chung nữa, mà chuyển sang học thuộc lòng (memorize) từng điểm dữ liệu, bao gồm cả các điểm nhiễu (noise/outliers). Hậu quả là mô hình đánh mất khả năng tổng quát hóa, dẫn đến hiệu năng trên tập kiểm định (Validation) không những không tăng mà còn sụt giảm.

4.1.2. Đề xuất các giải pháp khắc phục tiềm năng:

Để khắc phục hiện tượng Overfitting xảy ra với các mạng MLP sâu, ta có thể áp dụng các kỹ thuật tinh chỉnh (Regularization) sau đây:

- Dừng sớm (Early Stopping): Thay vì mù quáng huấn luyện đủ 50 epochs, ta sẽ viết code liên tục theo dõi đường Validation Loss. Ngay tại điểm phân kỳ (khi Validation Loss bắt đầu có dấu hiệu tăng lên liên tiếp trong 3-5 epochs), ta sẽ lập tức dừng quá trình huấn luyện và sử dụng trọng số tại thời điểm đó làm mô hình cuối cùng.
- Áp dụng L2 Regularization (Weight Decay): Bổ sung tham số phạt vào hàm suy hao bằng cách khai báo tham số weight_decay bên trong PyTorch Optimizer (ví dụ: optim.Adam(..., weight_decay=1e-4)). Điều này ép các hệ số weights phải duy trì ở giá trị nhỏ, hạn chế việc mạng nơ-ron trở nên quá "nhạy cảm" với dữ liệu huấn luyện.
- Thêm lớp Dropout (Dropout Layers): Chèn các lớp nn.Dropout(p=0.5) vào giữa các lớp ẩn Linear. Dropout sẽ tắt ngẫu nhiên 50% số nơ-ron trong mỗi bước lan truyền xuôi, ép các nơ-ron còn lại không được ỷ lại vào nhau và phải tự học được các đặc trưng mang tính cốt lõi, độc lập, giúp chống Overfitting cực kỳ mạnh mẽ.
- Gia tăng dữ liệu (Data Augmentation / Thu thập thêm): Khó khăn gốc rễ của bài toán là tập dữ liệu Diabetes quá nhỏ. Nếu có thể, hãy thu thập thêm bệnh án, hoặc áp dụng các kỹ thuật sinh dữ liệu nhân tạo (SMOTE) để làm phong phú tập huấn luyện.

4.2. Khảo sát Không không gian Đặc trưng học được



4.2.1. Cách thức trích xuất đầu ra của tầng ẩn:

Đầu ra của tầng ẩn được biểu diễn bằng phương trình toán học h = ReLU(W1x + b1). Để trích xuất thông tin này bằng PyTorch, trong lúc thiết kế hàm forward() của cấu trúc mạng (class BaselineMLP), tôi đã chủ động yêu cầu mô hình trả về (return) một tuple gồm hai thành phần: kết quả đầu ra cuối cùng (logits) và vector biến đổi sau khi đã đi qua hàm kích hoạt phi tuyến (biến h_relu). Ở quá trình suy luận (model.eval()), ta chỉ việc lấy ma trận của tập Validation truyền qua mô hình là có thể thu được toàn bộ không gian đặc trưng ẩn h của dữ liệu.

4.2.2. Phân tích sự thay đổi số chiều dữ liệu (Dimensionality Change):

Ở tầng ẩn thứ nhất, mô hình đã thực hiện một phép biến đổi từ không gian gốc x thuộc R^8 (tương ứng với 8 chỉ số sức khỏe ban đầu) sang một siêu không gian mới rộng lớn hơn h thuộc R^64. Việc "bơm" (project) dữ liệu từ 8 chiều lên 64 chiều giúp mở rộng số lượng các trục tọa độ. Nhờ số chiều lớn hơn, mạng nơ-ron có nhiều không gian (độ tự do) hơn để kéo giãn, xoắn ốc và phân bố lại các điểm dữ liệu.

4.2.3. Đánh giá khả năng phân tách trong không gian mới:

Dựa vào phép chiếu nén chiều PCA (từ không gian cao về mặt phẳng 2D) ở đồ thị trên, ta có thể rút ra kết luận quan trọng về tác dụng của Mạng nơ-ron:

- Ở không gian đầu vào (Original Space): Các điểm xanh (không tiểu đường) và các điểm đỏ (bị tiểu đường) nằm đan xen, chồng chéo hỗn loạn lên nhau. Ở không gian này, sẽ là bất khả thi nếu ta dùng một bộ phân lớp tuyến tính (Linear Classifier) như Logistic Regression để vẽ một đường thẳng chia cắt hoàn hảo hai lớp.
- Ở không gian ẩn học được (Learned Hidden Space): Sau khi trải qua biến đổi ma trận W1 và hàm phi tuyến ReLU, mô hình đã tự động tìm ra được các đặc trưng đại diện tốt hơn. Các điểm cùng màu (cùng nhãn) có xu hướng tụ tập lại với nhau thành các cụm (cluster) rõ rệt hơn, và vùng ranh giới giữa màu Xanh và Đỏ trở nên rành mạch hơn rất nhiều so với ban đầu.

⇒ Kết luận cốt lõi: Lớp ẩn (Hidden Layer) của mạng nơ-ron đóng vai trò như một bộ trích xuất đặc trưng tự động (Automated Feature Extractor). Nhiệm vụ của nó là bóp méo và nắn lại hình học của bộ dữ liệu gốc sao cho ở không gian mới, dữ liệu trở nên có thể phân tách tuyến tính (linearly separable). Lúc lúc này, lớp output layer (chỉ đóng vai trò như một Logistic Regression) có thể dễ dàng vạch một siêu mặt phẳng (hyperplane) cắt ngang để đưa ra dự đoán phân loại chính xác

4.3. So sánh và kết luận thực nghiệm tổng thể:

4.3.1. Bảng tổng hợp kết quả của 3 thí nghiệm:

Tiêu chí	Thí nghiệm 1 (Baseline MLP)	Thí nghiệm 2 (Deeper MLP)	Thí nghiệm 3 (SGD Optimization)
Kiến trúc mạng	8 -> 64 -> 2	8 -> 64 -> 32 -> 2	8 -> 64 -> 2
Số tham số (Parameters)	706	2,722	706
Hàm tối ưu (Optimizer)	Adam	Adam	SGD (Momentum = 0.9)
Tốc độ học (Learning Rate)	0.005	0.003	0.005
Số Vòng lặp (Epochs)	50	50	50
Tình trạng hội tụ	Khớp tốt / Hơi Overfitting nhẹ	Overfitting rõ rệt	Chậm nhưng ổn định
Độ chính xác Train (Train Acc)	92.51%	96.74%	88.44%
Độ chính xác Validation (Val Acc)	82.47%	83.77%	85.71%
Điểm F1-Score (Test Metrics)	0.81	0.83	0.85

(Lưu ý: Mức độ chính xác Validation (Val Acc) chính là con số quan trọng nhất phản ánh hiệu năng của mô hình).

4.3.2. Đánh giá mối quan hệ khoa học:

Từ những dữ liệu định lượng trên, ta có thể rút ra 3 kết luận mang tính quy luật về mối quan hệ giữa độ sâu, thuật toán tối ưu và biểu diễn đặc trưng trong mạng nơ-ron:

- Về mối quan hệ giữa Độ sâu của mạng (Network Depth) và Khả năng tổng quát hóa: Theo lý thuyết, mạng càng sâu (như ở Deeper MLP) thì khả năng xấp xỉ các hàm toán học phức tạp càng lớn. Tuy nhiên, thực nghiệm đã chứng minh một chân lý cốt lõi trong Học máy: "Mô hình tốt nhất không phải là mô hình phức tạp nhất, mà là mô hình phù hợp với kích thước dữ liệu nhất". Với một bộ dữ liệu dạng bảng nhỏ (tabular data) chỉ gần 800 quan sát như Diabetes, việc sử dụng 2,722 tham số của Thí nghiệm 2 đã gây ra hiện tượng học vẹt (Overfitting). Ngược lại, Thí nghiệm 1 với độ sâu vừa phải (706 tham số) mang lại tính ổn định trên tập dữ liệu chưa nhìn thấy (Validation/Test) tốt hơn hẳn.
- Về Tốc độ tối ưu hóa (Optimization Speed): Thực nghiệm số 3 cho thấy sự đánh đổi (trade-off) giữa các bộ tối ưu hóa. Adam (ở Thí nghiệm 1 và 2) đại diện cho tốc độ (Speed) – nó giúp hàm Loss lao dốc cực nhanh nhờ việc tự động thích ứng với từng gradient. SGD (Thí nghiệm 3) đại diện cho sự thận trọng và ổn định – tốc độ giảm loss chậm hơn rất nhiều, nhưng nó giúp mô hình tiếp cận các điểm cực tiểu (minima) phẳng hơn, qua đó đôi khi đạt được hiệu năng tổng quát hóa tốt hơn nếu huấn luyện đủ lâu.
- Về Chất lượng biểu diễn đặc trưng (Feature Representation): Khi so sánh với các thuật toán truyền thống như Logistic Regression (không có tầng ẩn), toàn bộ 3 mạng MLP này đều chứng minh sự ưu việt tuyệt đối trong việc trích xuất đặc trưng. Lớp ẩn ReLU(W1x + b1) hoạt động như một cỗ máy bẻ cong không gian, biến dữ liệu bị rối rắm, nhiễu loạn ở không gian gốc R^8 trở nên gọn gàng, chia cụm rõ ràng trong siêu không gian R^64. Điều này chứng minh rằng chất lượng dự đoán của mạng nơ-ron không nằm ở hàm đầu ra, mà nằm ở sức mạnh nắn chỉnh không gian hình học của các tầng ẩn.

PHẦN V: TỔNG KẾT VÀ DANH MỤC SẢN PHẨM

5.1. Kết luận chung

Thông qua việc thiết kế, cài đặt và phân tích 3 thí nghiệm đối chứng với mạng Multi-Layer Perceptron (MLP) trên tập dữ liệu Diabetes, thực nghiệm đã mang lại những bài học kinh nghiệm quý giá sau:

5.1.1. Sức mạnh và giới hạn của mạng nơ-ron (Neural Networks):

- Sức mạnh chuyển đổi không gian: Không giống như các mô hình học máy cơ bản (như Logistic Regression), mạng nơ-ron thực sự mạnh mẽ ở khả năng dùng các tầng ẩn (Hidden layers) kết hợp hàm phi tuyến (ReLU) để tự động nắn chỉnh, bóp méo không gian dữ liệu gốc. Phân tích PCA ở mục 4.2 đã chứng minh rõ: dữ liệu vốn không thể phân tách tuyến tính ở không gian 8 chiều ban đầu đã được sắp xếp lại thành các cụm rõ rệt và dễ phân loại hơn trong không gian 64 chiều.
- Sự nhạy cảm với dữ liệu (Data Hungry): Bài học lớn nhất rút ra là Mạng nơ-ron cực kỳ "đói" dữ liệu. Khi áp dụng mô hình MLP (dù là mạng cơ sở hay mạng sâu) lên một tập dữ liệu y tế khá nhỏ (~800 mẫu), hiện tượng Quá khớp (Overfitting) xuất hiện rất nhanh và rõ rệt. Điều này cho thấy thuật toán Deep Learning không phải là giải pháp vạn năng. Với dữ liệu dạng bảng nhỏ, đôi khi các mô hình truyền thống (như Random Forest, XGBoost) lại cho độ ổn định cao hơn và ít tốn kém chi phí tinh chỉnh hơn.

5.1.2. Đánh đổi trong thiết kế cấu trúc (Architecture Trade-offs):

- Thí nghiệm 2 (Deeper MLP) đã chứng minh rằng: Nhiều lớp hơn không có nghĩa là tốt hơn. Việc tăng số lượng nơ-ron và độ sâu đột ngột làm tăng số tham số từ 706 lên 2,722, khiến mạng quá phức tạp so với độ khó của bài toán. Thay vì học quy luật chung, mô hình lại đi học thuộc các nhiễu (noise) trong dữ liệu, dẫn đến Validation Loss tăng ngược trở lại. Kinh nghiệm là luôn bắt đầu bằng một mạng đơn giản nhất có thể (Baseline) và chỉ tăng độ sâu khi mạng bị Underfitting.

5.1.3. Tầm quan trọng của việc chọn Thuật toán Tối ưu hóa (Optimizers):

- So sánh giữa Adam và SGD ở Thí nghiệm 3 cho thấy không có một bộ tối ưu hóa nào là hoàn hảo cho mọi bài toán. Adam giúp mô hình hội tụ cực kỳ nhanh, phù hợp cho việc rèn luyện các mô hình lớn trong thời gian ngắn, nhưng đôi khi dễ bị kẹt ở điểm Overfitting hoặc các cực tiểu hẹp. Trong khi đó, SGD kết hợp với Momentum, tuy tốc độ học chậm rạp hơn, nhưng lại mang đến một quỹ đạo

hội tụ ổn định và kiên định, giúp mô hình bám sát quỹ đạo đi xuống của loss mà không bị dao động quá mạnh.

⇒ Tổng kết lại: Quá trình huấn luyện một mạng nơ-ron thành công không chỉ nằm ở việc viết đúng cú pháp code PyTorch, mà là một quá trình thực nghiệm khoa học. Người kỹ sư AI phải liên tục quan sát đồ thị học tập (Learning curves), hiểu rõ sự thay đổi biến dạng ma trận Tensor đi qua từng lớp (Forward pass), và khéo léo tinh chỉnh sự cân bằng giữa dung lượng mô hình (Capacity) và độ lớn của dữ liệu để mô hình đạt được khả năng tổng quát hóa cao nhất.